

Taking (natural) logarithms,

$$-\epsilon \sum_{t=1}^T m_i^{(t)} \leq \ln(N) + \epsilon^2 T - \epsilon \sum_{t=1}^T \vec{p}^{(t)} \cdot \vec{m}^{(t)}$$

and the final result follows by dividing by ϵ and rearranging the terms. ■

Remark The above proof may seem messy with all the inequalities. However, it follows a standard “framework”: upper and lower bound the potential function. These Multiplicative Weight Update type of algorithms are most often analyzed using this potential argument. Once you know that cool argument, the analysis is almost automatic modulo some rewriting.

For the application of solving covering LPs, it will be convenient to consider the *average cost* incurred per day. We also generalize so that the cost vector can take values in $[-\rho, \rho]^N$ where ρ is called the “width”. The proof is the same by scaling and we get the following corollary:

Corollary 3 *Suppose $\epsilon \leq 1$ and for $t \in [T]$, $\vec{p}^{(t)}$ is picked by Hedge and cost vectors satisfy $\vec{m}^{(t)} \in [-\rho, \rho]^N$. If $T \geq (4\rho^2 \ln N)/\epsilon^2$, then for any expert i :*

$$\frac{1}{T} \sum_{t=1}^T \vec{p}^{(t)} \cdot \vec{m}^{(t)} \leq \frac{1}{T} \sum_{t=1}^T m_i^{(t)} + 2\epsilon.$$

4 Hedging for LPs

We now turn to a nice application of the Hedge method to that of solving *covering LPs*. We start by defining these LPs.

4.1 Covering LPs

Definition 4 *A linear program of the form :*

$$\begin{aligned} \text{minimize} \quad & \sum_{j=1}^n c_j x_j \\ \text{subject to} \quad & Ax \geq b \\ & 1 \geq x_j \geq 0 \quad \forall j \end{aligned}$$

is a covering linear program if

$$A \in \mathbb{R}_+^{m \times n}, \quad b \in \mathbb{R}_+^m \quad \text{and} \quad c \in \mathbb{R}_+^n$$

This definition simply ensures that all the coefficients of the constraints and of the objective function are non-negative. Notice that both the set cover relaxation and the vertex cover relaxation that we saw in class were covering LPs.

Let us now introduce an example of a covering LP that we will reuse later:

$$\begin{aligned} \text{minimize} \quad & x_1 + 2x_2 \\ \text{subject to} \quad & x_1 + 3x_2 \geq 2 \\ & 2x_1 + x_2 \geq 1 \\ & 1 \geq x_1, x_2 \geq 0 \end{aligned} \tag{1}$$

4.2 General idea

The idea of using the Hedge method for linear programming is to associate an expert with each constraint of the LP. In other words, the Hedge method will maintain a weight distribution over the set of constraints of a linear problem to solve, and to iteratively update those weights in a multiplicative manner based on the cost function at each step/day. Of course we need to define the cost function in a smart way later but let us first define the notion of an oracle and then give a small instructive example.

Initially, the Hedge method will give a weight $w_i^{(1)} = 1$ for every constraint/expert $i = 1, \dots, m$ (the number m of constraints now equals the number of experts). And at each day t , it will maintain a convex combination $\vec{p}^{(t)}$ of the constraints (that is defined in terms of the weights). Using such a convex combination $\vec{p}$, a natural easier LP with a single constraint is obtained by summing up all the constraints according to $\vec{p}$. Any optimal solution of the original LP is also a solution of this reduced problem, so the new problem will have at most the same cost as the previous one. We define an oracle for solving this reduced problem:

Definition 5 *An oracle that, given $\vec{p} = (p_1, \dots, p_m) \geq \mathbf{0}$ such that $\sum_{i=1}^m p_i = 1$, outputs an optimal solution x^* to the following reduced linear problem:*

$$\begin{aligned} \text{minimize} \quad & \sum_{j=1}^n c_j x_j \\ \text{subject to} \quad & \left(\sum_{i=1}^m p_i A_i \right) \cdot x \geq \sum_{i=1}^m p_i b_i \\ & 1 \geq x \geq 0 \end{aligned}$$

This linear problem is in practice much easier to solve than the original one and we can design a simple greedy algorithm for the oracle.

For concreteness, let us consider a small example. If we apply the method we described to our example (1), we have two initial weights $w_1 = w_2 = 1$ and thus $p_1 = p_2 = \frac{1}{2}$, and we sum all the constraints:

$$\begin{aligned} p_1(x_1 + 3x_2) + p_2(2x_1 + x_2) &\geq p_1 \cdot 2 + p_2 \cdot 1 \Leftrightarrow \\ 1.5x_1 + 2x_2 &\geq 1.5 \\ 1 &\geq x_1, x_2 \geq 0 \end{aligned}$$

By using the oracle, an optimal solution to this reduced problem is $x_1 = 1, x_2 = 0$ of cost 1. But is this a feasible solution to our original problem? By checking the constraints of the original LP:

$$\begin{array}{ll} 2x_1 + x_2 = 2 \geq 1 & \text{OK} \\ x_1 + 3x_2 = 1 < 2 & \text{not OK} \end{array}$$

We will need to go back to the original problem and *increase the weights of the unsatisfied constraints* to give them more importance and *decrease the weights of the satisfied constraint*. We will perform these updates (as done by Hedge) multiplicatively that we describe in detail in Section 4.4. We first describe how to implement the oracle in general.

4.3 Implementation of the oracle

The oracle is given an objective function **minimize** $\sum_{i=1}^n c_i x_i$ and only one constraint that we can rewrite as $\sum_{i=1}^n d_i x_i \geq b$ (which is the weighted sum of all constraints). We also have $1 \geq x_i \geq 0 \forall i$ and $c_i, d_i \geq 0 \forall i$ since it is a covering problem.

The idea is to assign the maximum value (namely 1) to the variables that have the highest ratio *constraint coefficient/objective coefficient*. That way we will satisfy the constraint as fast as possible while maintaining a small objective function. Then assign zero to the other variables and possibly an intermediate value for the variable that is at the limit to make the constraint satisfied. This is very similar to the most “bang-for-the-buck” greedy and it is the solution to fractional knapsack. Formally:

- Sort and relabel all variables x_i so that $d_1/c_1 \geq d_2/c_2 \geq \dots d_n/c_n$.
- Let $k = \min\{j : \sum_{i=1}^j d_i \geq b\}$ and $\ell = b - \sum_{i=1}^{k-1} d_i$.
- Set $x_i := 1$ for $i = 1, \dots, k-1$.
- Set $x_k := \ell/d_k$.
- Set $x_i := 0$ for $i = k+1, \dots, n$.

We have,

$$\sum_{i=1}^n d_i x_i = \sum_{i=1}^n d_i x_i = \sum_{i=1}^{k-1} d_i x_i + d_k x_{\sigma(k)} + \sum_{i=k+1}^n d_i x_i = \sum_{i=1}^{k-1} d_i + \ell = b$$

Therefore the constraint is exactly satisfied. By the ordering of the variables, this ensures that we minimize the objective function as required. Having implemented the oracle, we proceed to formally define the Hedge algorithm for linear programming.

4.4 Hedge algorithm for covering LPs

As already explained, we associate an expert to each constraint of the covering LP. In addition, as hinted above, we wish to increase the weight of unsatisfied constraints and decrease the weight of satisfied constraints (in a smooth manner depending on the size of the violation or the slack). The Hedge algorithm for covering LPs thus becomes:

- Assign each constraint i a weight $w_i^{(1)}$ initialized to 1.

At each time t :

- Pick the distribution $p_i^{(t)} = w_i^{(t)} / \Phi^{(t)}$ where $\Phi^{(t)} = \sum_{i \in [N]} w_i^{(t)}$.
- Now we define the cost vector instead of the adversary as follows:
 - Let $x^{(t)}$ be the solution returned by the oracle on the LP obtained by using the convex combination $\bar{p}^{(t)}$ of constraints. Notice that cost of $x^{(t)}$, i.e., $c^\top x^{(t)}$, is at most the cost of an optimal solution to the original LP.
 - Define the cost of constraint i as

$$m_i^{(t)} = \sum_{j=1}^n A_{ij} x_j - b_i = A_i x - b_i.$$

Notice that we have a positive cost if the constraint is satisfied (so the weight will be decreased by Hedge) and a negative cost if it is violated (so the weight will be increased by Hedge).

- After observing the cost vector, set $w_i^{(t+1)} = w_i^{(t)} \cdot e^{-\varepsilon \cdot m_i^{(t)}}$.

Output: the average $\bar{x} = \frac{1}{T} \sum_{t=1}^T x^{(t)}$ of the constructed solutions.

Analysis. Since the analysis for the Hedge algorithm works with an adversarial construction of the cost vectors, it definitely holds for the cost vectors we constructed. Let

$$\rho = \max_{1 \leq i \leq m} \{\max(b_i, A_i \mathbf{1} - b_i)\},$$

be (an upper bound on) the width of our constructed cost vectors. By Corollary 3, we thus have for $\epsilon \in [0, 1], T \geq (4\rho^2 \ln m)/\epsilon^2$ and any constraint i that

$$\frac{1}{T} \sum_{t=1}^T \vec{p}^{(t)} \cdot \vec{m}^{(t)} \leq \frac{1}{T} \sum_{t=1}^T m_i^{(t)} + 2\epsilon. \quad (2)$$

Let us first consider the left-hand-side of the above expression. We have that

$$\begin{aligned} \sum_{t=1}^T \vec{p}^{(t)} \cdot \vec{m}^{(t)} &= \sum_{t=1}^T \left(\sum_i p_i^{(t)} \cdot m_i^{(t)} \right) \\ &= \sum_{t=1}^T \left(\underbrace{\sum_i p_i^{(t)} \cdot (A_i x^{(t)} - b_i)}_{(*)} \right) \end{aligned}$$

which is non-negative because $(*)$ is non-negative for every t since the oracle outputs a feasible solution. Using that the left-hand-side of (2) is non-negative, we get

$$-2\epsilon \leq \frac{1}{T} \sum_{t=1}^T m_i^{(t)} = \frac{1}{T} \sum_{t=1}^T (A_i x^{(t)} - b_i) = A_i \bar{x} - b_i$$

which implies that

$$A_i \bar{x} \geq b_i - 2\epsilon,$$

for every constraint i . So our solution $\bar{x}$ is **almost feasible**. Moreover the cost $c^\top \bar{x} = c^\top (\frac{1}{T} \sum_{t \in [T]} x^{(t)})$ is **at most the cost of an optimal solution** to the original LP since each $x^{(t)}$ has no higher cost than an optimal solution (by the properties of the oracle).

Setting the parameters for solving the set cover relaxation. For set cover we have that $\rho \leq n$ and therefore, it is sufficient to set $T = (4n^2 \ln m)/\epsilon^2$ (this can in fact be improved by a better analysis to $\approx n \ln m/\epsilon^2$). This gives a solution $\bar{x}$ that satisfies $\sum_{e \in S} x_e \geq 1 - 2\epsilon$ for every set S and the cost of $\bar{x}$ is at most that of an optimal LP solution. We can obtain a feasible (approximately optimal) solution by simply defining $x^* = \frac{1}{1-2\epsilon} \bar{x}$.